James Newbegin (104563879)

SWE30009 – Assignment 3

Task 1:

1.1:

Program Variables:

k = 3

D = 550

1st Partition Variables:

- d = 100
- m = 3
- n = 35
- θ = 3/100 = 0.3

2nd Partition Variables:

- d = 200
- m = 5
- n = 35
- θ = 5/200 = 0.025

3rd Partition Variables:

- d = 250
- m = 2
- n = 35
- θ = 2/250 = 0.008

Sampling Rates:

35/100 = 0.35

35/200 = 0.175

35/250 = 0.14

Random Testing Variables:

d = 550

m = 10

n = 99

θ = 10/550 = 0.018

σ = 99/550 = 0.18

P-measure:

Random Testing:

$Pr=1-(1-\theta)n$

Pr = 1 – (1 – 10/550)^99

Pr = 0.8374166894

Partition Testing:

$Pp=1-k\prod i=1k(1-\theta i)ni$

Pp = 1 - (1 - 0.3)^20(1 - 0.025)^20(1 - 0.008)^20

Pp = 0.9995904692

E-measure:

Random Testing:

Er = nθ

Er = 99*0.018

Er = 1.8

Partition Testing:

$Ep = k\sum i=1ni\theta i$

Ep = 20*3/100 + 20*5/200 + 20*2/250

Ep = 1.26

F-measure:

Random Testing:

Fr = 1/θ

Fr = d/m

Fr = 55

Partition Testing:

$Fp = k\sum i=1(1/\theta i)*(ni/n)$

Fp = 100/3*20/99 + 200/5*20/99 + 250/2*20/99

Fp = 40.06734007

The P-measure for the partition testing is very high, with it displaying that there is a 99% chance of detecting at least one failure. This highlights that partition testing is more reliable than random testing for this program, considering the P-measure for random testing is 84%.

The proportional sampling strategy selecting test cases randomly from partitions that have the same sampling rate. This is done by finding values for the number of test cases that divide by each partition's domain size to result in the same value, such as with 18, 36 and 45 for the first, second and third partition respectively. This method guarantees that the P-measure of the partition testing with it is not less than that of random testing, and that the E-measure of both are equal. This means that it is more effective than random testing demonstrated in the comparison below.

Number of Test Cases:

Partition Testing (PSS):

- 1st Partition: 16
- 2nd Partition: 36
- 3rd Partition: 45

Partition Testing:

- 1st Partition: 33
- 2nd Partition: 33
- 3rd Partition: 33

Random Testing:

- 99

	P-measure	E-measure	F-measure
Partition Testing (PSS)	0.99	1.8	77
Partition Testing	0.99	1.26	40
Random Testing	0.84	1.8	55

1.2:

Program 1: Returns games based on attributes (tags, sorting, price, search)

The test oracle is a mechanism or procedure to check whether the output for any input in the input domain is correct or not. Considering this, applying the test oracle for this program would be too time consuming and as a result metamorphic testing is appropriate. To complete this, metamorphic relations are found by finding methods to predictably alter the FO according to property changes. There is opportunity for this in the program, with four attributes which can provide predictable results from change as detailed below.

MR1: If the SO is found by searching for a string, then the FO is a subset of the SO if a character is added to the end of the string searched for such that FO ⊆ SO and will contain the same or less values.

MR2: If the SO is found by using the tags, then the FO is a subset of the SO if another tag is added such that FO ⊆ SO and will contain the same or less values.

MR3: Regardless of the type of sorting used to display results FO = SO if the other attributes are the same.

MR4: Considering the price attribute takes a minimum and maximum value, if the maximum value is reduced for the FO while the minimum value remains the same, the FO will be a subset of the SO such that FO ⊆ SO and will contain the same or less values.

These metamorphic relations can be tested with the source and follow-up inputs:

SI:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $5 - $100
Search: "Had"

FI1:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $5 - $100
Search: "Hade"

FI2:

Tags: Roguelike, 2D, Singleplayer, 8-bit
Sorting: Highest price to lowest price
Price: $5 - $100
Search: "Had"

FI3:

Tags: Roguelike, 2D, Singleplayer
Sorting: A-Z
Price: $5 - $100
Search: "Had"

FI4:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $5 - $20
Search: "Had"

MR1 would be violated should FO1 return more results than SO, MR2 would be violated should FO2 return more results than SO, MR3 would be violated should FO3 return a different number of results to SO, and MR4 would be violated should FO4 return more results than SO.

A set of metamorphic groups (a pair of a source input and follow-up input) can also be created to test a single MR. For MR1 the following metamorphic groups can be made:

MG1:

SI:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $100-$105
Search: "Had"

FI:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $100-$102
Search: "Had"

MG2:

SI:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $40-$200
Search: "Had"

FI:

Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $40-$150

Search: "Had"

MG3:
SI:
Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $5 - $20
Search: "Had"

FI:
Tags: Roguelike, 2D, Singleplayer
Sorting: Highest price to lowest price
Price: $5 - $10
Search: "Had"

The results are as follows:
SO1 = 30 results
SO2 = 15 results
SO3 = 20 results
FO1 = 31 results
FO2 = 40 results
FO3 = 20 results

Considering 2 of the 3 metamorphic groups violated the metamorphic relation, a violation rate can be drawn from this. This is 2/3 for this example, as it is denoted by:
'The number of MGs in the set that violate the MR / number of MGs' or 'V/N'
A higher violation rate can be seen as positive, as it shows a higher proficiency of revealing faults, which shows a better test case selection strategy.

Program 2: Finds the product of a non-empty set of integers

This program's outputs are predictable, with metamorphic relations demonstrating this as shown below.
MR1: If the first element of the SI is multiplied by 10, then FO = 10 * SO
MR2: If the first element of the SI is halved, then FO = SO / 2
MR3: If the SI is split into multiple arrays, where all the numbers from the SI in their same amounts can be found in the arrays, then where FO is the product of the results of those arrays, FO = SO
From these MRs, the following metamorphic groups can be created.

MG1:
SI: [6, 3, 5, 7, 9]
FI: [60, 3, 5, 7, 9]

MG2:
SI: [6, 3, 5, 7, 9]
FI: [3, 3, 5, 7, 9]

MG3:
SI: [6, 3, 5, 7, 9]
FI: [6, 3, 5], [7, 9]

The results are as follows:
SO1: 5670
FO1: 56700
SO2: 5670
FO2: 2835 (2835 * 2 = 5670)
SO3: 5670
FO3: 5670 (6 * 3 * 5 = 90, 7 * 9 = 63, 63 * 90 = 5670)

Considering the results it can be said that none of the MRs were violated, with a violation rate of 0%. This indicates a poor test case selection strategy or robustness of the program.

1.3:

The program outlined below will be considered for mutant testing.
C = A+B*3
The following table can be constructed based on the program and the test inputs [A=2, B=3] denoted by TC1, [A=2, B=2] denoted by TC2, [A=3, B=3] denoted by TC3 and [A=1, B=1] denoted by TC4. TC1 and TC2 belong to test suite TS1, while TC3 and TC4 belong to test suite TS2. The table involves mutation operators, with the "+" and "*" both being replaced by other operators to generate mutants. Variables and constants were also changed, with "A" being replaced with "3" and "3" being replaced with "A" in mutants M9-M17.

Mutant ID	Mutant Program	TC1 (TS1)	TC2 (TS1)	TC3 (TS2)	TC4 (TS2)
Given P	C=A+B*3	11	8	12	4
M1	C=A*B*3	18	12	27	1
M2	C=A*B+3	9	7	12	4

M3	C=A*B-3	3	1	6	-2
M4	C=A+B+3	8	7	9	5
M5	C=A+B-3	2	1	3	-1
M6	C=A-B*3	-7	-4	-6	-2
M7	C=A-B+3	2	3	3	3
M8	C=A-B-3	-4	-3	-3	-3
M9	C=3+A*B	9	7	12	4
M10	C=3*A+B	9	8	12	4
M11	C=3*A-B	3	4	6	2
M12	C=3+A+B	8	7	9	5
M13	C=3+A-B	2	3	3	3
M14	C=3-A*B	-3	-1	-6	2
M15	C=3-A+B	4	3	3	3
M16	C=3-A-B	-2	-1	-3	1

Mutants can be killed, should their output differ from P's output, demonstrating the test case's proficiency to differenciate between mutants and the program. Considering this, from this table it can be told that TS1 is a better test suite than TS2 as TS1 has a mutant score of 94% killing 16/17 mutants while TS2 has a mutant score of 75% killing 12/16 mutants.

Another program can be considered for testing as seen below.

```
BEGIN
    IF (A > B && B <= 3)
        RETURN true
    END IF
END
```

Another table can be constructed using this program and the test cases [A=4, B=3] denoted by TS1, [A=2, B=2] denoted by TC2, [A=3, B=3] denoted by TC3 and [A=3, B=2] denoted by TC4. TC1 and TC2 belong to test suite TS1, while TC3 and TC4 belong to test suite TS2. Further mutation operators are used for this table, with the change of relational and logical operators in the code.

Mutant ID	Mutant Program	TC1 (TS1)	TC2 (TS1)	TC3 (TS2)	TC4 (TS2)
Given P	A > B && B <= 3	True	False	False	True
M1	A <= B && B <= 3	False	True	True	False
M2	A <= B && B < 3	False	True	False	False
M3	A <= B && B >= 3	False	False	True	False
M4	A <= B && B > 3	False	False	False	False
M5	A < B && B <= 3	False	False	False	False
M6	A < B && B < 3	False	False	False	False
M7	A < B && B >= 3	False	False	False	False
M8	A < B && B > 3	False	False	False	False
M9	A >= B && B <= 3	True	True	True	True
M10	A >= B && B < 3	False	True	False	True
M11	A >= B && B >= 3	True	False	True	False
M12	A >= B && B > 3	False	False	False	False
M13	A > B && B <= 3	True	False	False	True

M14	A > B && B < 3	False	False	False	True
M15	A > B && B >= 3	True	False	False	False
M16	A > B && B > 3	False	False	False	False
M17	A <= B || B <= 3	True	True	True	True
M18	A <= B || B < 3	False	True	True	True
M19	A <= B || B >= 3	True	True	True	False
M20	A <= B || B > 3	False	True	True	False
M21	A < B || B <= 3	True	True	True	True
M22	A < B || B < 3	False	True	False	True
M23	A < B || B >= 3	True	False	True	False
M24	A < B || B > 3	False	False	False	False
M25	A >= B || B <= 3	True	True	True	True
M26	A >= B || B < 3	True	True	True	True
M27	A >= B || B >= 3	True	True	True	True
M28	A >= B || B > 3	True	True	True	True
M29	A > B || B <= 3	True	True	True	True
M30	A > B || B < 3	True	True	False	True
M31	A > B || B >= 3	True	False	True	True
M32	A > B || B > 3	True	False	False	True

Unexpectedly, it is told from the table that TS1 had a mutation score of 50%, with 16/32 mutants killed, and TS2 also had a mutation score of 50% with 16/32 mutants killed. This demonstrates a poor test case selection strategy.

Task 2: Test a program of your choice.

Program: Three Sum

This program involves taking an input of a set of numbers and finding groups of three numbers within the set that sum to 0. Considering the functionality of this program, it can be said that the outputs are predictable, and metamorphic and mutation testing is appropriate. There are numerous opportunities in the program's code to implement mutants. This can especially be seen with the lines below where the operators to be changed have been highlighted:

Lines 26-27:

```
while low < high:
        if nums[low] + nums[high] == c:
```

Lines 37 – 40:

```
elif nums[low] + nums[high] < c:
            low += 1
        else:
            high -= 1
```

Originally, the arithmetic operator on the 27th line "+" for "if nums[low] + nums[high] == c:" was not considered for change for mutation testing, and all four variations of the relational operator "<" on line 26 were included for mutation testing. However, the testing found that any input put through a mutant with either "while low > high:" or "while low >= high:" would return a empty array "[]". This was because prior to line 26 being ran, low and high are respectively set as the minimum and maximum index numbers for the input array. Consequently, the program will never execute past line 26 if it contains a ">" or ">=" as low is always initialized as a lower number than high. Considering the repetitive nature of the results ">" and ">=" would return when used on the 26th line, they were excluded from the mutation testing, with further tests developed using variations of the artithmetic operator on the 27th line.

The number of mutations that will be made for testing is found using the equation below.

2 * 3 * 2 * 2 * 2
= 2^4 * 3
= 16 * 3
= 48

These 48 mutations can also be used to achieve complete metamorphic testing of the program. This is outlined with the MRs below:

MR1: If FI is the SI with additional numbers, then FO ⊂ SO where SO does not output []

MR2: If FI is a permutation of SI, then FO = SO

MR1 can be tested with the metamorphic groups below.

MG1:
SI: [1, 0, -1]
FI: [1, 0, -1, 3]

MG2:
SI: [1, 2, -3]
FI: [1, 2, -3, 4]

MG3:
SI: [2, -7, 3, 4]
FI: [2, -7, 3, 4, 9]

MG4:
SI: [3, -3, 0]
FI: [3, -3, 0, 8]

MG5:
SI: [17, 13, -30]
FI: [17, 13, -30, 5]

Below is a table featuring all the metamorphic test groups tested against all the mutations, to display the effectiveness of the MR.

M. ID	Mutant Operators [26, 27(1), 27(2), 38, 40]	SI (MG 1)	FI (MG1)	SI (MG 2)	FI (MG2)	SI (MG3)	FI (MG3)	SI (MG4)	FI (MG4)	SI (MG5)	FI (MG5)
Given P	<, +, ==, +=, -=	[-1, 0, 1]	[-1, 0, 1]	[-3, 1, 2]	[-3, 1, 2]	[-7, 3, 4]	[-7, 3, 4]	[-3, 0, 3]	[-3, 0, 3]	[-30, 13, 17]	[-30, 13, 17]
M1	<, +, ==, +=, +=	[-1, 0, 1]	Error	[-3, 1, 2]	Error	Error	Error	[-3, 0, 3]	Error	[-30, 13, 17]	Error
M2	<, +, ==, -=, -=	[-1, 0, 1]	[-1, 0, 1]	[-3, 1, 2]	[-3, 1, 2]	Error	Error	[-3, 0, 3]	[-3, 0, 3]	[-30, 13, 17]	Error
M3	<, +, ==, -=, +=	[-1, 0, 1]	Error	[-3, 1, 2]	Error	Error	Error	[-3, 0, 3]	Error	[-30, 13, 17]	Error
M4	<, +, !=, +=, -=	[]	[-1, 0, 3], [0, 1, 3]	[]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [2, 3, 9], [3, 4, 9]	[]	[-3, 0, 8], [0, 3, 8]	[]	[-30, 5, 17], [5, 13, 17]
M5	<, +, !=, +=, +=	Error	[-1, 0, 3], [0, 1, 3]	[-3, 1, 4], [1, 2, 4]	Error	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 9], [2, 3, 9], [3, 4, 9]	Error	[-3, 0, 8], [0, 3, 8]	[-30, 5, 17], [5, 13, 17]	Error
M6	<, +, !=, -=, -=	[]	[-1, 0, 3], [0, 1, 3]	[]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [2, 3, 9], [3, 4, 9]	[]	[-3, 0, 8], [0, 3, 8]	[]	[-30, 5, 17], [5, 13, 17]

M7	<, +, !=, -=, +=	Error	[-1, 0, 3], [0, 1, 3]	Error	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 9], [2, 3, 9], [3, 4, 9]	Error	[-3, 0, 8], [0, 3, 8]	Error	[-30, 5, 17], [5, 13, 17]
M8	<=, +, ==, +=, -=	[-1, 0, 1]	[-1, 0, 1]	[-3, 1, 2]	[-3, 1, 2]	[-7, 3, 4]	[-7, 3, 4]	[-3, 0, 3]	[-3, 0, 3]	[-30, 13, 17]	[-30, 13, 17]
M9	<=, +, ==, +=, +=	[-1, 0, 1]	Error	[-3, 1, 2]	Error	Error	Error	[-3, 0, 3]	Error	[-30, 13, 17]	Error
M10	<=, +, ==, -=, -=	[-1, 0, 1]	[-1, 0, 1]	[-3, 1, 2]	[-3, 1, 2]	Error	Error	[-3, 0, 3]	[-3, 0, 3]	[-30, 13, 17]	Error
M11	<=, +, ==, -=, +=	[-1, 0, 1]	Error	[-3, 1, 2]	Error	Error	Error	[-3, 0, 3]	Error	[-30, 13, 17]	Error
M12	<=, +, !=, +=, -=	[-1, 0, 0]	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	[-3, 1, 1]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 3], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 0]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 13]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M13	<=, +, !=, +=, +=	Error	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	Error	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 9], [-7, 4, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	Error	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	Error	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M14	<=, +, !=, -=, -=	[-1, 0, 0]	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	[-3, 1, 1]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 3], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 0]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 13]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M15	<=, +, !=, -=, +=	Error	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	Error	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 9], [-7, 4, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	Error	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	Error	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M16	<, -, ==, +=, -=	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
M17	<, -, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M18	<, -, ==, -=, -=	[]	[]	[]	[]	Error	[-7, 9, 2]	[]	[]	[]	Error
M19	<, -, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M20	<, -, !=, +=, -=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]
M21	<, -, !=, +=, +=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]

M22	<, -, !=, -=, -=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]
M23	<, -, !=, -=, +=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]
M24	<=, -, ==, +=, -=	[]	[0, 1, 1]	[]	[]	[]	[]	[]	[0, 3, 3]	[]	[]
M25	<=, -, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M26	<=, -, ==, -=, -=	[-1, 1, 0], [-1, 0, -1]	[0, 1, 1]	Error	Error	Error	Error	[-3, 3, 0], [-3, 0, -3]	[0, 3, 3]	Error	Error
M27	<=, -, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M28	<=, -, !=, +=, -=	[-1, 0, 1]	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M29	<=, -, !=, +=, +=	[-1, 0, 1]	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M30	<=, -, !=, -=, -=	[-1, 0, 1]	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M31	<=, -, !=, -=, +=	[-1, 0, 1]	[-1, 0, 3], [-1, 1, 1], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M32	<, *, ==, +=, -=	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
M33	<, *, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M34	<, *, ==, -=, -=	[]	[]	[]	[]	Error	Error	[]	[]	[]	Error
M35	<, *, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M36	<, *, !=, +=, -=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2,	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]

							3, 9], [3, 4, 9]				
M37	<, *, !=, +=, +=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]
M38	<, *, !=, -=, -=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]
M39	<, *, !=, -=, +=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [1, 2, 4]	[-7, 2, 4], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [5, 13, 17]
M40	<=, *, ==, +=, -=	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
M41	<=, *, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M42	<=, *, ==, -=, -=	Error	[]	Error	Error	Error	Error	Error	[]	Error	Error
M43	<=, *, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M44	<=, *, !=, +=, -=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M45	<=, *, !=, +=, +=	[-1, 0, 1]	[-1, 0, 3], [-1, 1, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M46	<=, *, !=, -=, -=	[-1, 0, 1]	[-1, 0, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]
M47	<=, *, !=, -=, +=	[-1, 0, 1]	[-1, 0, 3], [-1, 1, 3], [0, 1, 3]	[-3, 1, 2]	[-3, 1, 4], [-3, 2, 2], [1, 2, 4]	[-7, 2, 4], [-7, 3, 3], [2, 3, 4]	[-7, 2, 9], [-7, 3, 4], [2, 3, 9], [2, 4, 4], [3, 4, 9]	[-3, 0, 3]	[-3, 0, 8], [-3, 3, 3], [0, 3, 8]	[-30, 13, 17]	[-30, 5, 17], [-30, 13, 13], [5, 13, 17]

MG3 and MG5 were the most successful, with a mutation score of 97.9% with 46/47 mutants killed. The other metamorphic groups achieved a similar mutation score of 93.6% with 44/47 mutants killed. This 5% difference is because of the line “elif nums[low] + nums[high] < c:” not being true for MG1, MG2 and MG4, allowing the inputs to be processed normally and output a result the same as the given P.
MR2 can be tested with the metamorphic groups below.

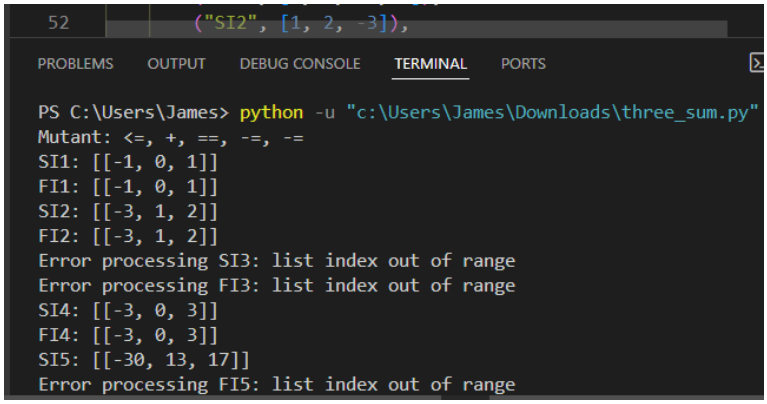
MG1:
SI: [-2, 0, 2]
FI: [2, 0, -2]

MG2:
SI: [3, 1, 4, 5]
FI: [5, 4, 3, 1]

MG3:
SI: [-4, 4, 0]
FI: [0, 4, -4]

MG4:
SI: [-10, 7, 3]
FI: [3, 7, -10]

MG5:
SI: [99, 1, 99]
FI: [1, 99, 99]



Below is a table featuring all the metamorphic test groups tested against all the mutations, to display the effectiveness of the MR.

M. ID	Mutant Operators [26, 27(1), 27(2), 38, 40]	SI (MG1)	FI (MG1)	SI (MG2)	FI (MG2)	SI (MG3)	FI (MG3)	SI (MG4)	FI (MG4)	SI (MG5)	FI (MG5)
Given P	<, +, ==, +=, -=	[-2, 0, 2]	[-2, 0, 2]	[]	[]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[]	[]
M1	<, +, ==, +=, +=	[-2, 0, 2]	[-2, 0, 2]	Error	Error	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	Error	Error
M2	<, +, ==, -=, -=	[-2, 0, 2]	[-2, 0, 2]	[]	[]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[]	[]
M3	<, +, ==, -=, +=	[-2, 0, 2]	[-2, 0, 2]	Error	Error	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	Error	Error
M4	<, +, !=, +=, -=	[]	[]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[]	[]	[]	[]	[1, 99, 99]	[1, 99, 99]
M5	<, +, !=, +=, +=	Error	Error	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	Error	Error	Error	Error	[1, 99, 99]	[1, 99, 99]
M6	<, +, !=, -=, -=	[]	[]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[]	[]	[]	[]	[1, 99, 99]	[1, 99, 99]
M7	<, +, !=, -=, +=	Error	Error	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	Error	Error	Error	Error	[1, 99, 99]	[1, 99, 99]
M8	<=, +, ==, +=, -=	[-2, 0, 2]	[-2, 0, 2]	[]	[]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[]	[]
M9	<=, +, ==, +=, +=	[-2, 0, 2]	[-2, 0, 2]	Error	Error	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	Error	Error
M10	<=, +, ==, -=, -=	[-2, 0, 2]	[-2, 0, 2]	[]	[]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[]	[]
M11	<=, +, ==, -=, +=	[-2, 0, 2]	[-2, 0, 2]	Error	Error	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	Error	Error
M12	<=, +, !=, +=, -=	[-2, 0, 0]	[-2, 0, 0]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 0]	[-4, 0, 0]	[-10, 3, 3]	[-10, 3, 3]	[1, 99, 99]	[1, 99, 99]

M13	<=, +, !=, +=, +=	Error	Error	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	Error	Error	Error	Error	[1, 99, 99]	[1, 99, 99]
M14	<=, +, !=, -=, -=	[-2, 0, 0]	[-2, 0, 0]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 0]	[-4, 0, 0]	[-10, 3, 3]	[-10, 3, 3]	[1, 99, 99]	[1, 99, 99]
M15	<=, +, !=, -=, +=	Error	Error	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	Error	Error	Error	Error	[1, 99, 99]	[1, 99, 99]
M16	<, -, ==, +=, -=	[]	[]	[1, 3, 4]	[1, 3, 4]	[]	[]	[]	[]	[]	[]
M17	<, -, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M18	<, -, ==, -=, -=	[]	[]	[1, 3, 4]	[1, 3, 4]	[]	[]	[]	[]	[]	[]
M19	<, -, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M20	<, -, !=, +=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M21	<, -, !=, +=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M22	<, -, !=, -=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M23	<, -, !=, -=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M24	<=, -, ==, +=, -=	[]	[]	[1, 3, 4]	[1, 3, 4]	[]	[]	[]	[]	[]	[]
M25	<=, -, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M26	<=, -, ==, -=, -=	[-2, 2, 0], [-2, 0, -2]	[-2, 2, 0], [-2, 0, -2]	[1, 3, 4]	[1, 3, 4]	[-4, 4, 0], [-4, 0, -4]	[-4, 4, 0], [-4, 0, -4]	Error	Error	[]	[]
M27	<=, -, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M28	<=, -, !=, +=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M29	<=, -, !=, +=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M30	<=, -, !=, -=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M31	<=, -, !=, -=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M32	<, *, ==, +=, -=	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
M33	<, *, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error

M34	<, *, ==, -=, -=	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
M35	<, *, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M36	<, *, !=, +=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M37	<, *, !=, +=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M38	<, *, !=, -=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M39	<, *, !=, -=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [3, 4, 5]	[1, 3, 5], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M40	<=, *, ==, +=, -=	[]	[]	[]	[]	[]	[]	[]	[]	[]	[]
M41	<=, *, ==, +=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M42	<=, *, ==, -=, -=	Error	Error	[]	[]	Error	Error	Error	Error	[]	[]
M43	<=, *, ==, -=, +=	Error	Error	Error	Error	Error	Error	Error	Error	Error	Error
M44	<=, *, !=, +=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M45	<=, *, !=, +=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M46	<=, *, !=, -=, -=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]
M47	<=, *, !=, -=, +=	[-2, 0, 2]	[-2, 0, 2]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[1, 3, 5], [1, 4, 4], [3, 4, 5]	[-4, 0, 4]	[-4, 0, 4]	[-10, 3, 7]	[-10, 3, 7]	[1, 99, 99]	[1, 99, 99]

MG2 was the best with 85.1% and 40/47 mutants killed, followed by MG5 with 76.7% with 36/47, while the other MGs performed poorly, with a score of 51.1%.

In conclusion, MR1 was found to be highly effective. This is seen with how only mutations 2, 8 and 10 returned a result the same as the given P for the test cases, reflecting a robustness in the code. Conversely, MR2s findings saw values identical to the given P for many of the test cases, demonstrating a poor test case selection strategy. This is especially seen with MG1, MG2 and MG4 pitted against mutations involving a "!=" on line 27, with them failing as should any group of 3 numbers found in the array be output, it would equal their given P. Furthermore, certain mutations unexpectedly returned errors, revealing potential problematic code, such as with mutations 25, 27, 33, 35, 41, 43.

